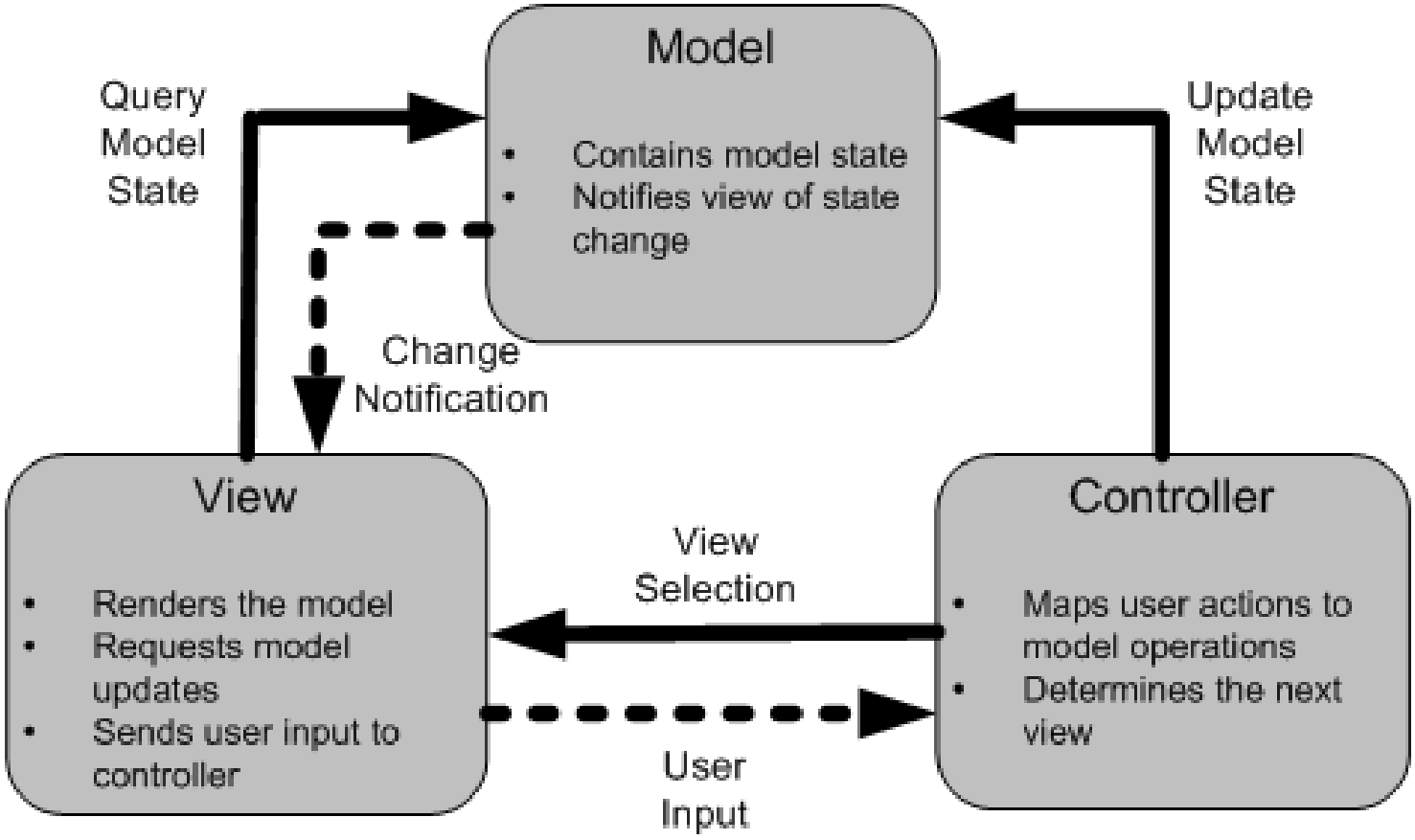
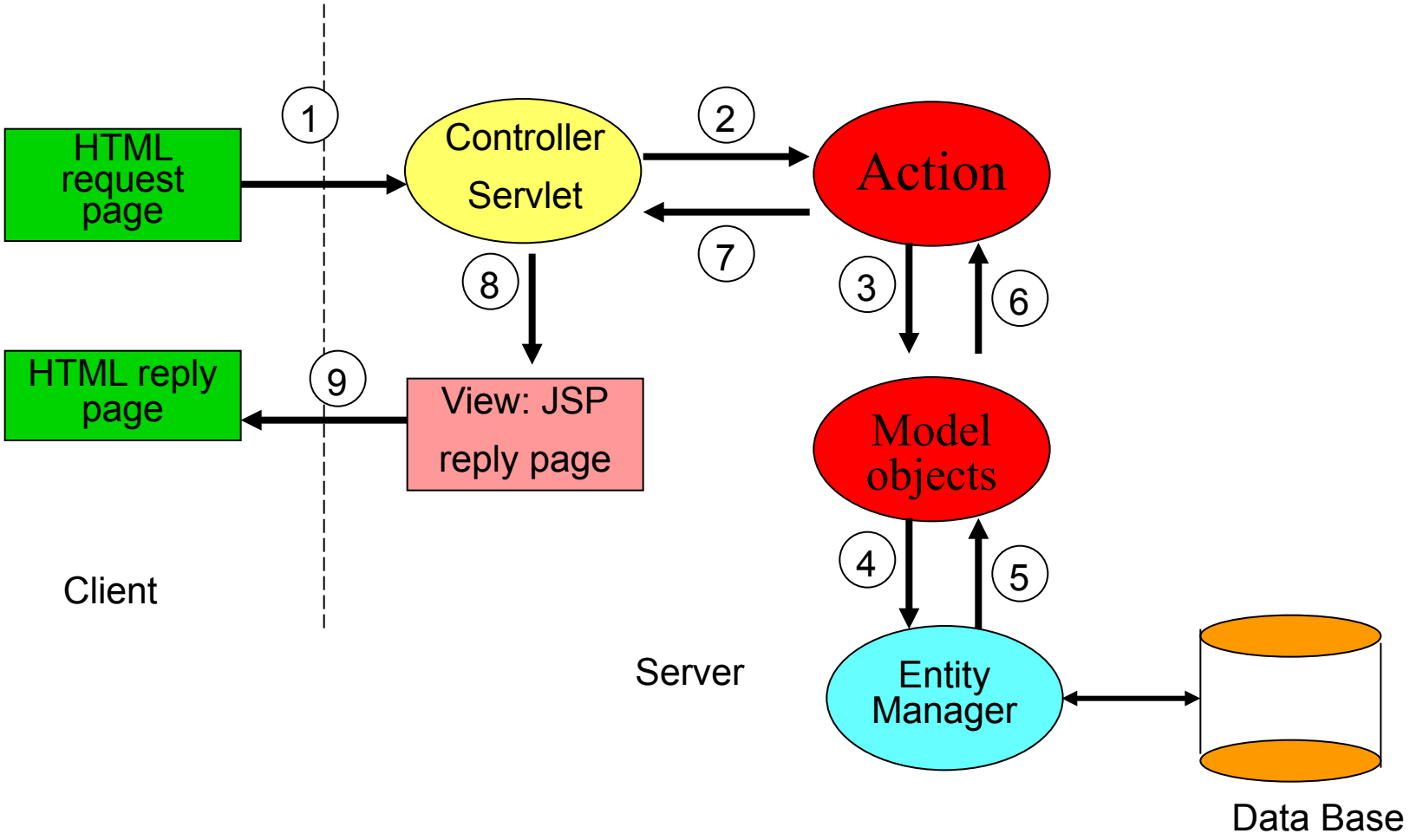
MVC pattern

Juan Luis Gorricho

Web applications design pattern



Service schema



Actions

- All actions implement the interface Action:

```
public interface Action {
    public abstract void perform(HttpServletRequest req,
    HttpServletResponse resp) throws Exception;
}
```

- Any implementation of “perform” method has for last sentence:

```
ViewManager.nextView(req, resp, "/view/next_view.jsp");
```

ViewManager

- The ViewManager is a static class forwarding the request to the next view:

```
public static void nextView(… req, … resp, String jsp) {
    try {
        RequestDispatcher dispatcher = req.getRequestDispatcher(jsp);
        if (dispatcher != null) {
            dispatcher.forward(req, resp);
        }
    } catch (Exception e) {
        System.out.println("Error at dispatcher in nextView " + e.toString());
    }
}
```

ControllerServlet

- On web.xml all actions (*.do) are mapped to the ControllerServlet
- The init() method on the ControllerServlet creates a HashMap adding all available actions
- The doPost or doGet identifies and performs the corresponding action

ContextListener

- The ContextListener retrieves the **EntityManager** and **UserTransaction** references and creates those objects with application scope: **CategoryModel** and **ProductModel** in charge of accessing the DDBB
- The objects with application scope are accessible thru the **ServletContext**

Actions

- Actions to be coded:
  - initAction: shows all categories
  - categoryAction: shows all items of one category
  - neworderAction: adds a new item to the shopping cart
  - viewcartAction: shows the shopping cart
  - updatecartAction: updates the shopping cart items
  - clearcartAction: removes all items from the shopping cart
  - checkoutAction: performs the buy action

Web pages

- The web pages directory contains all JSPs with the views and any complementary files, in particular:
  - img subdirectory with all images,
  - view subdirectory with all view JPSs
  - 3 complementary JSP files on the root directory: start.jsp, error.jsp and undone.jsp.

ShoppingCart

- Methods:

public synchronized void addItem(Product product)

public synchronized void update(Product product, int quantity)

public synchronized List<ShoppingCartItem> getItems()

public synchronized int getNumberOfItems()

public synchronized double getTotal()

public synchronized void clear()

ShoppingCartItem

- Methods:

public ShoppingCartItem(Product product)

public Product getProduct()

public int getQuantity()

public void setQuantity(int quantity)

public double getTotal()

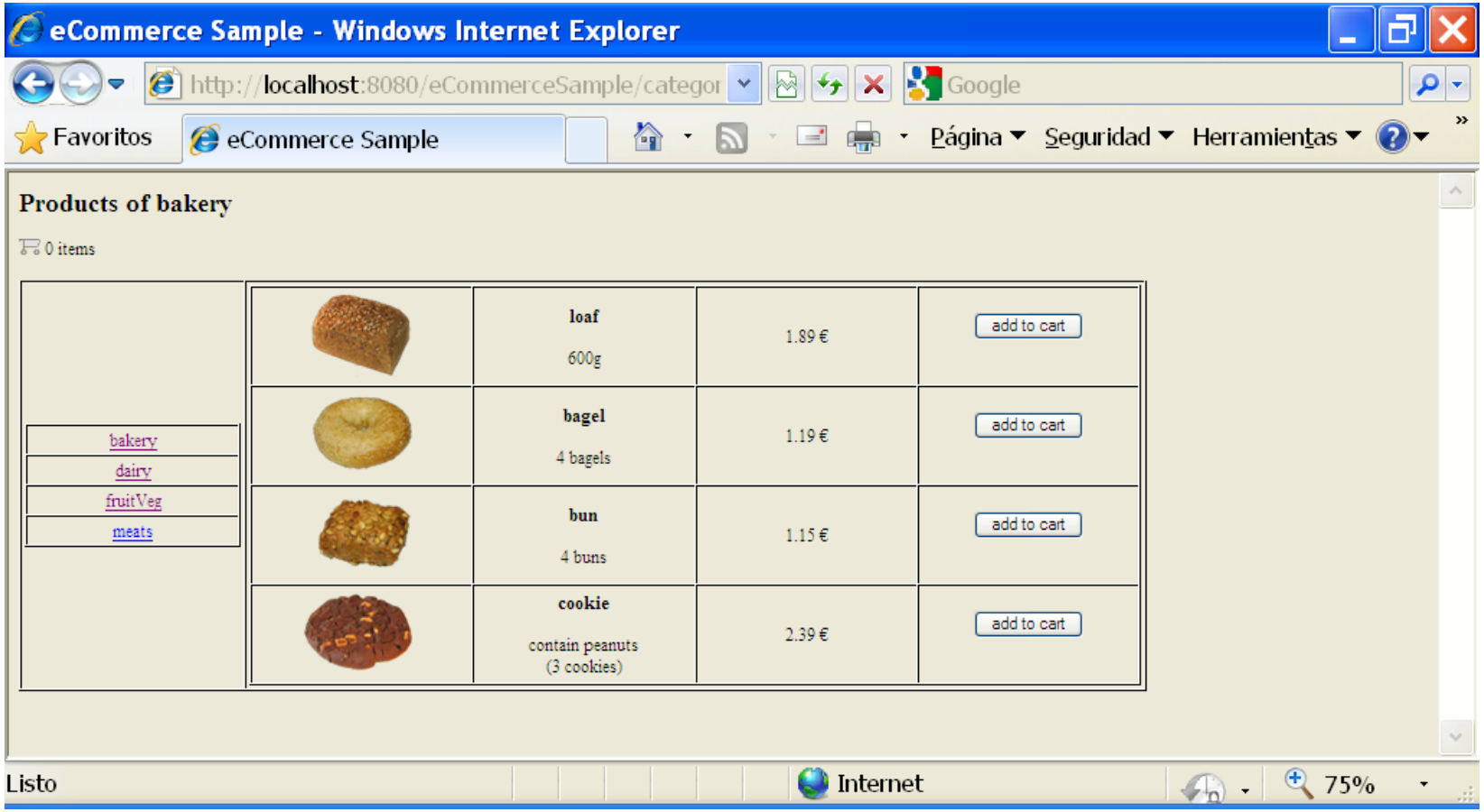
Exercise

- Download the ecommerce_exercise
- Restore the ecommerce.sql database script
- Open the ecommerce_exercise from NetBeans and set up the database connection as requested
- Deploy the web application and check the access to the initial page of the virtual grocery market

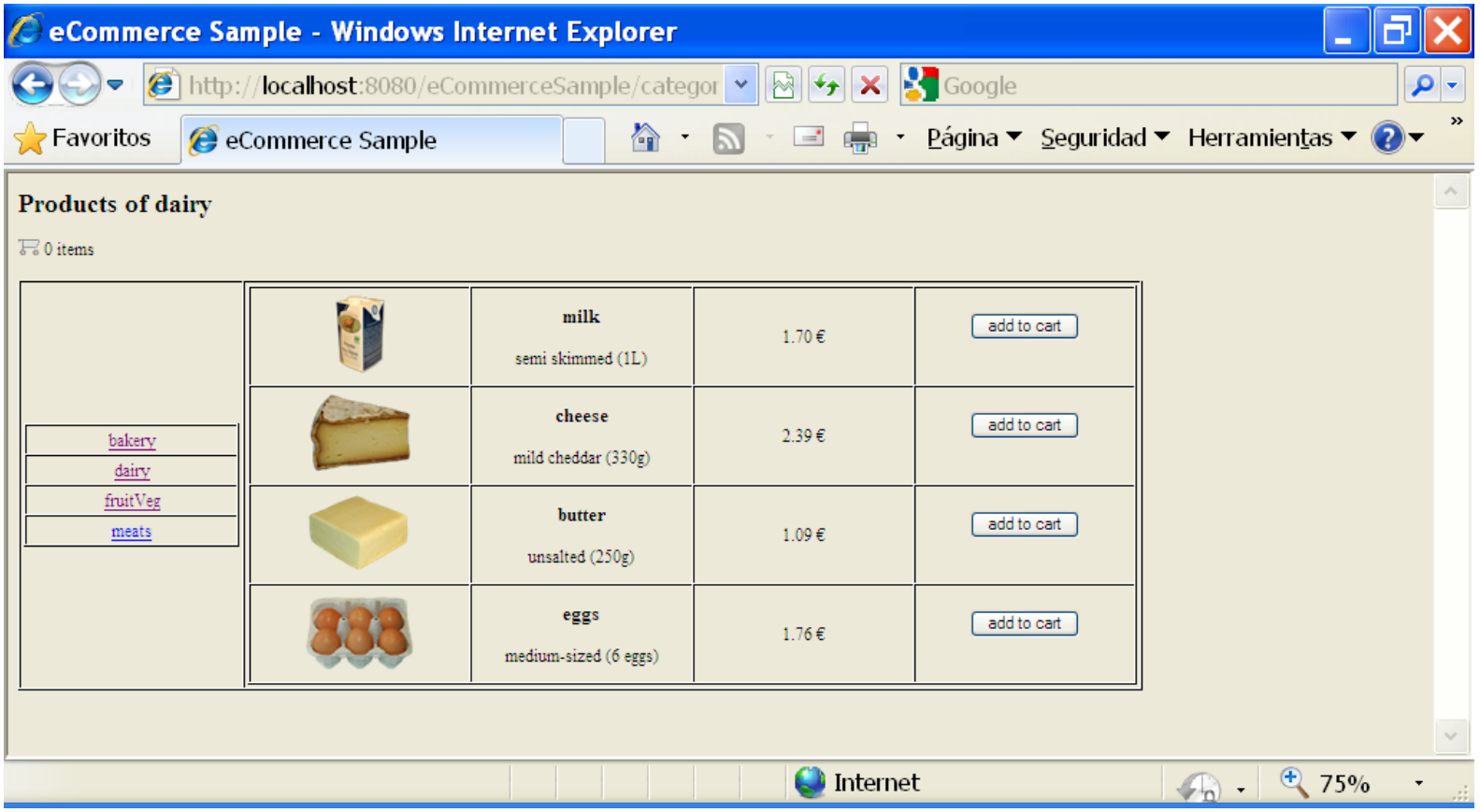
eCommerce portal prototype



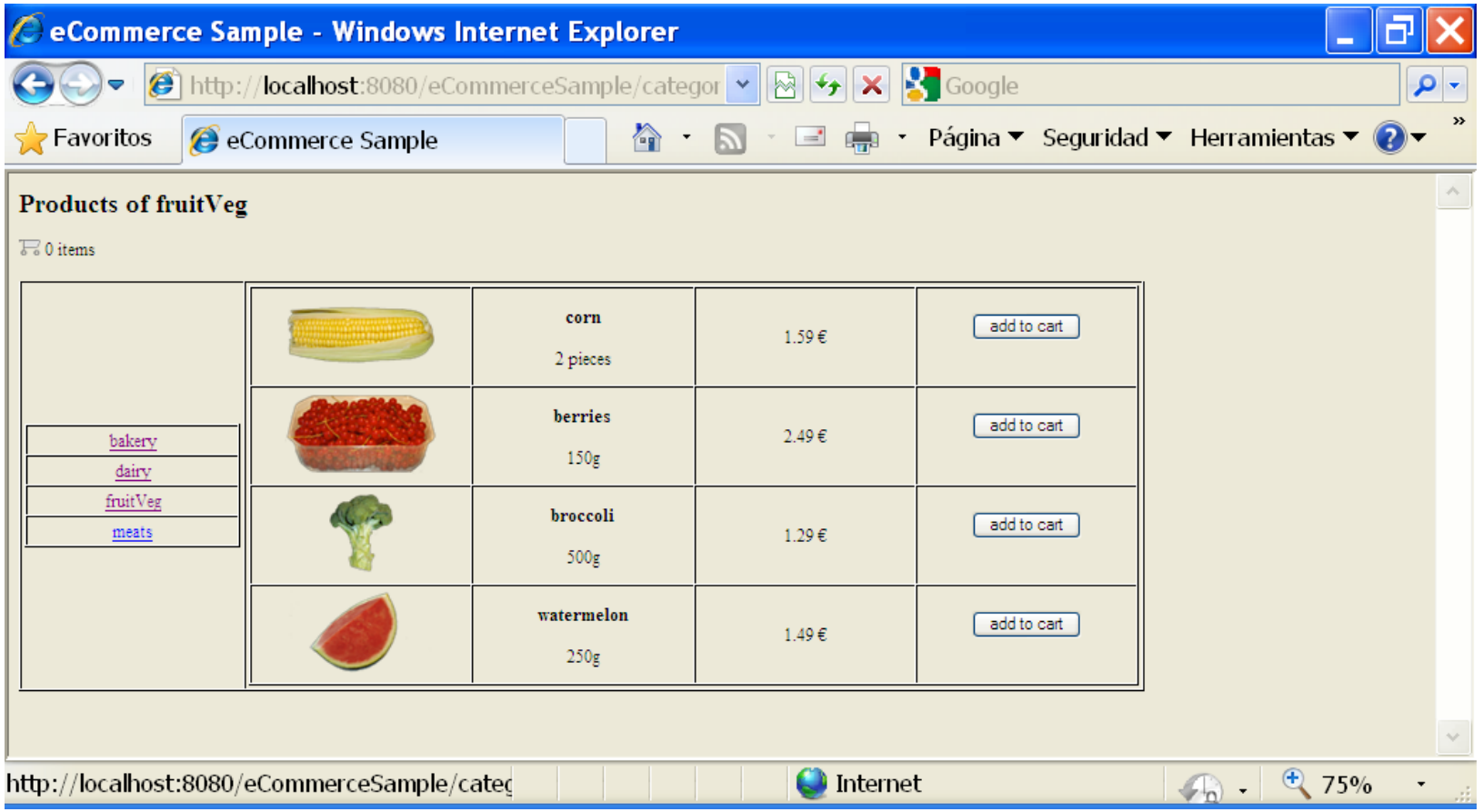
eCommerce portal prototype



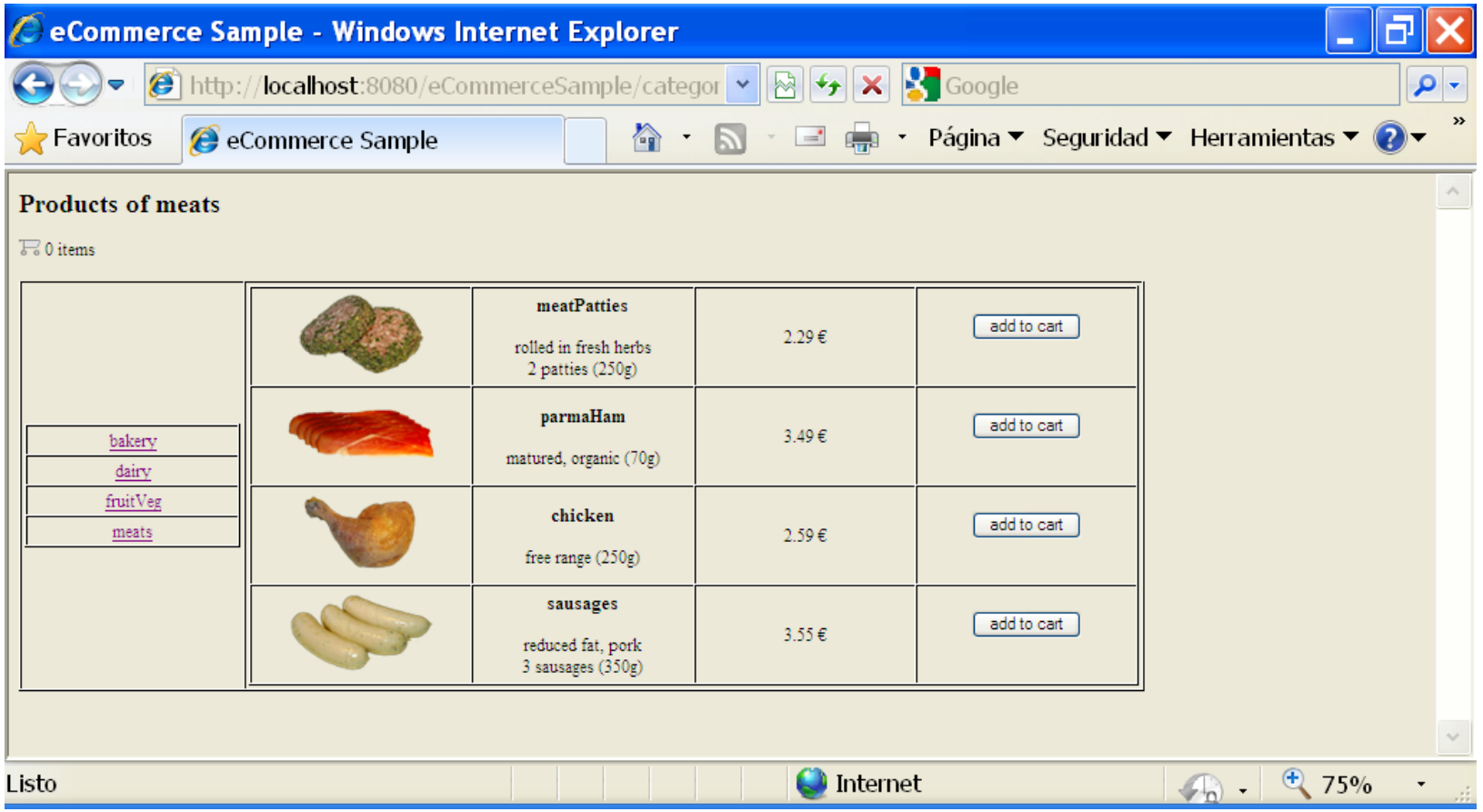
eCommerce portal prototype



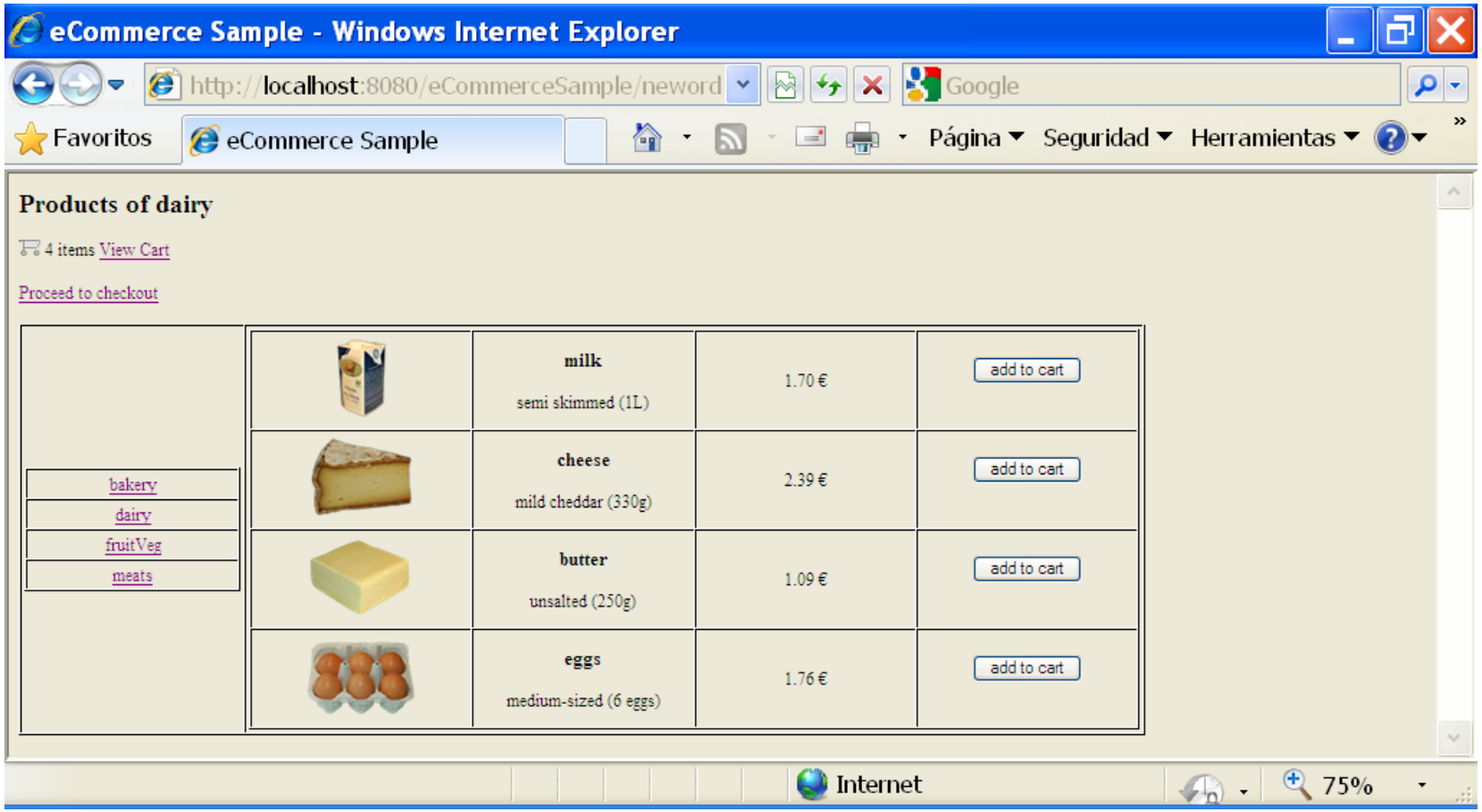
eCommerce portal prototype



eCommerce portal prototype



eCommerce portal prototype



eCommerce portal prototype

